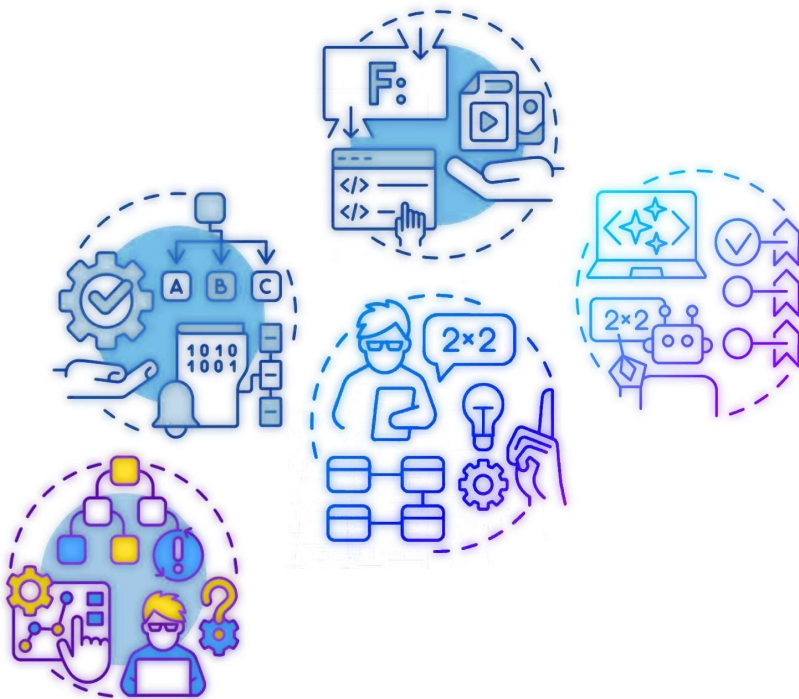




Paradigma Orientado a Objetos

Esp. Ing. Viviana A. Santucci
Ing. Jimena Bourlot
Ing. Exequiel Benavídez
Ing. Omar Cristian Martínez

Tecnologías de la Programación

Repaso Principios SOLID

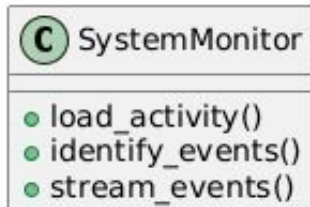
- **Single Responsibility Principle (SRP):** una clase debe tener una única responsabilidad. Separar funciones y clases según tareas específicas
- **Open/Closed Principle (OCP):** el código debe estar abierto a la extensión, pero cerrado a la modificación. Usar herencia o composición para extender funcionalidad sin modificar el código base.
- **Liskov Substitution Principle (LSP):** los objetos de una subclase deben poder reemplazar a los de la clase base sin romper el programa. Las clases que implementan una interfaz o heredan de otra deben respetar el comportamiento esperado.

Repaso Principios SOLID

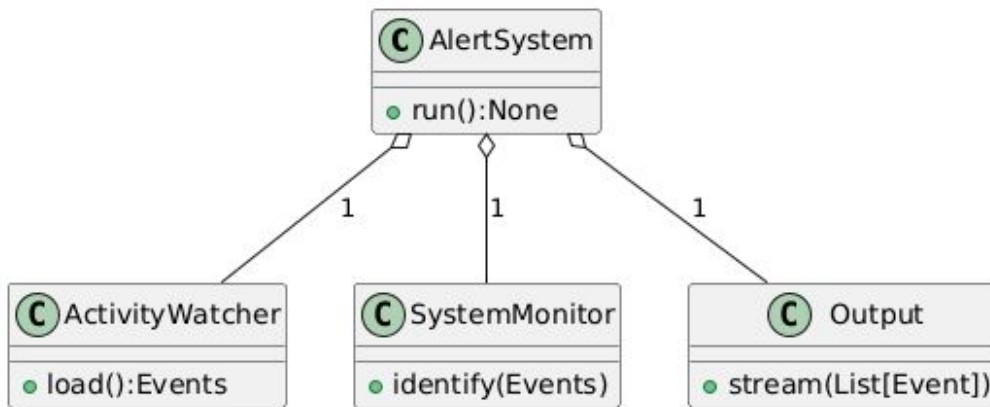
- **Interface Segregation Principle (ISP):** los clientes no deberían estar forzados a depender de interfaces que no usan.
- **Dependency Inversion Principle (DIP):** los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de abstracciones.

Single Responsibility Principle (SRP)

Una clase debe tener una única responsabilidad. Separar funciones y clases según tareas específicas



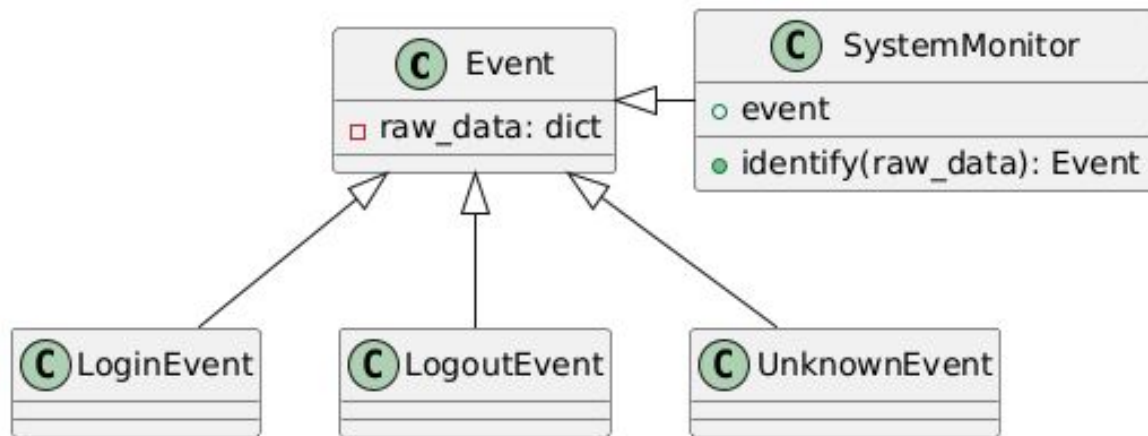
Diseño que **no**
cumple el
principio SRP



Diseño **cumple**
el principio SRP

Open/Closed Principle (OCP)

El código debe estar abierto a la extensión, pero cerrado a la modificación. Usar herencia o composición para extender funcionalidad sin modificar el código base.



Enfoque **incorrecto**

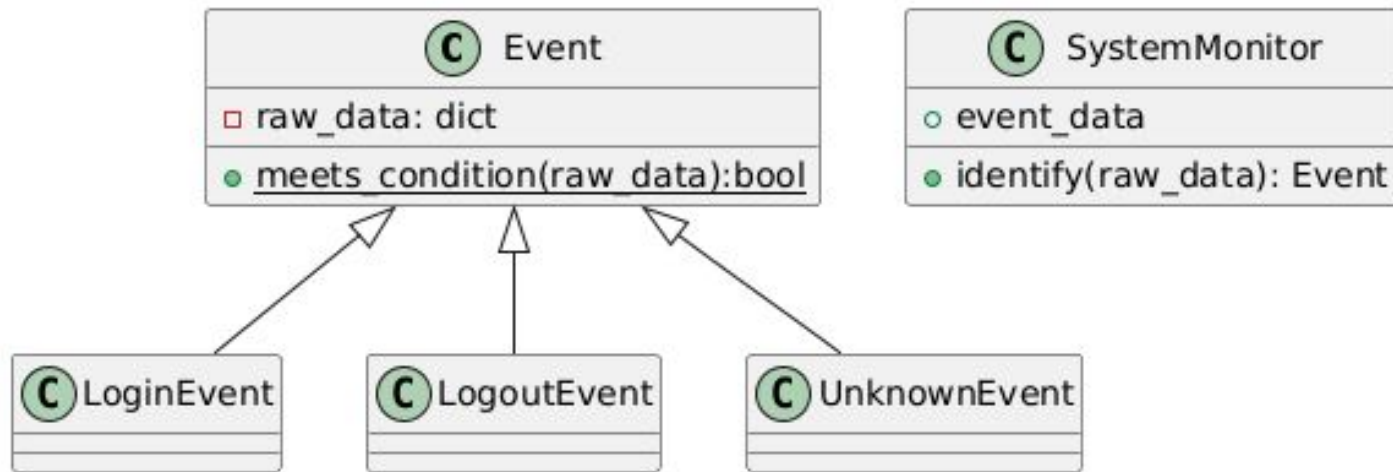
Este sistema es demasiado rígido, ya que el método “identify” tiene que chequear todas las opciones para saber qué evento fue el que ocurrió.

Open/Closed Principle (OCP)

Podemos ver que extender el sistema es muy complicado.

```
def identify_event(self):  
    if (  
        self.event_data["before"]["session"] == 0  
        and self.event_data["after"]["session"] == 1  
    ):  
        return LoginEvent(self.event_data)  
    elif (  
        self.event_data["before"]["session"] == 1  
        and self.event_data["after"]["session"] == 0  
    ):  
        return LogoutEvent(self.event_data)  
  
    return UnknownEvent(self.event_data)
```

Open/Closed Principle (OCP)



Enfoque **correcto**

En este caso “Event” tiene el método “meets_condition” que debe ser implementado por sus clases hijas.

Open/Closed Principle (OCP)

En este caso el método es mucho más genérico y facilita su extensión.

```
def identify_event(self):  
    for event_cls in Event.__subclasses__():  
        try:  
            if event_cls.meets_condition(self.event_data):  
                return event_cls(self.event_data)  
        except KeyError:  
            continue  
    return UnknownEvent(self.event_data)
```

Liskov Substitution Principle (LSP)

Los objetos de una subclase deben poder reemplazar a los de la clase base sin romper el programa. Las clases que implementan una interfaz o heredan de otra deben respetar el comportamiento esperado.

Liskov Substitution Principle (LSP)

```
class Event:
    ...
    def meets_condition(self, event_data: dict) -> bool:
        return False

class LoginEvent(Event):
    def meets_condition(self, event_data: list) -> bool:
        return bool(event_data)
```

```
class LogoutEvent(Event):
    def meets_condition(self, event_data: dict, override: bool) ->
bool:
    if override:
        return True
    ...
```

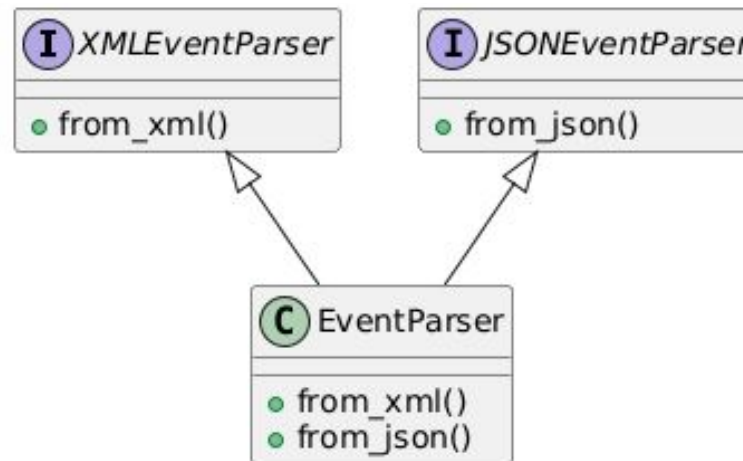
Casos incorrectos. En este caso porque las clases hijas implementan los métodos cambiando los argumentos, por esto no podrían ser reemplazados por una clases superior.

Interface Segregation Principle (ISP)

Los clientes no deberían estar forzados a depender de interfaces que no usan.



Enfoque
incorrecto
o



Enfoque
correcto

Interface Segregation Principle (ISP)

Podemos ver que las interfaces son más pequeñas.

Para este principio debemos pensar en que

“The smaller the interface, the better”

```
from abc import ABCMeta, abstractmethod

class XMLEventParser(metaclass=ABCMeta):
    @abstractmethod
    def from_xml(xml_data: str):
        """Parse an event from a source in XML representation."""

class JSONEventParser(metaclass=ABCMeta):
    @abstractmethod
    def from_json(json_data: str):
        """Parse an event from a source in JSON format."""

class EventParser(XMLEventParser, JSONEventParser):
    """An event parser that can create an event from source data either
    in XML or JSON format.
    """

    def from_xml(xml_data):
        pass

    def from_json(json_data: str):
        pass
```

Dependency Inversion Principle (DIP)

Los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de abstracciones.

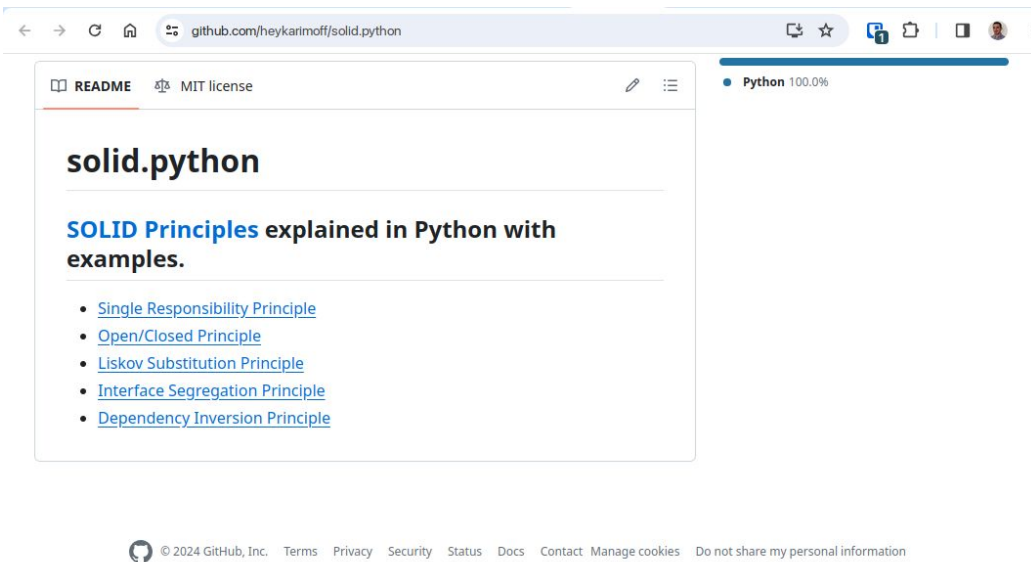
En el primer caso si cambiamos algo en la forma que queremos enviar la información a Syslog debemos adecuar EventStreamer también.



Dependency Inversion Principle (DIP)

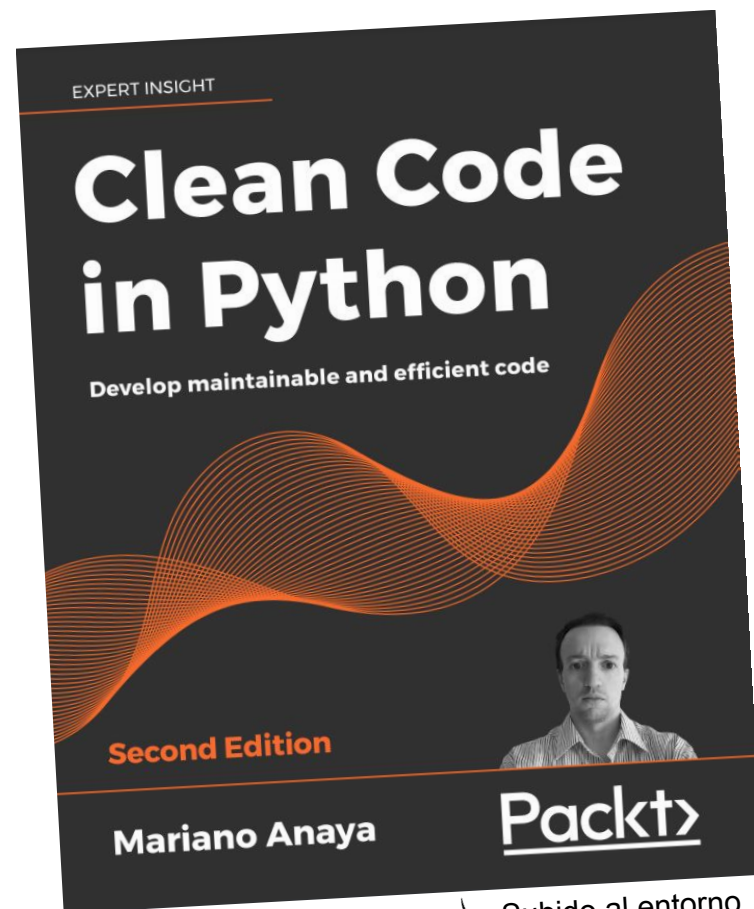
Sin embargo realizando la implementación a través de una interfaz, abstraemos a EventStreamer de posibles cambios en la forma del envío de información.





<https://github.com/heykarimoff/solid.python>

<https://www.linkedin.com/pulse/solid-design-principles-python-examples-hiral-amodia/>



Subido al entorno

Ley de Demeter

“Habla sólo con tus amigos”

No se debe llamar a métodos de los objetos devueltos por otros métodos.

Se deben evitar cosas del estilo:

```
a.obtener_x().obtener_y().obtener_valor()
```

En su cambio reemplazar por funciones del estilo:

```
a.obtener_xy_valor()
```

Ley de Demeter

El siguiente código cumple con Demeter?

```
class Cuenta:
    def __init__(self, cliente):
        self.cliente = cliente
    def obtener_pais_cliente(self):
        return self.cliente.obtener_pais()

class Cliente:
    def __init__(self, direccion):
        self.direccion = direccion
    def obtener_pais(self):
        return self.direccion.obtener_pais()

class Direccion:
    def __init__(self, pais):
        self.pais = pais
    def obtener_pais(self):
        return self.pais

cuenta = Cuenta(Cliente(Direccion("Argentina")))
pais = cuenta.obtener_pais_cliente()
```

Ley de Demeter

El siguiente código cumple con Demeter?

```
class Cuenta:
    def __init__(self, cliente):
        self.cliente = cliente
    def obtener_pais_cliente(self):
        return self.cliente.obtener_pais()

class Cliente:
    def __init__(self, direccion):
        self.direccion = direccion
    def obtener_pais(self):
        return self.direccion.obtener_pais()

class Direccion:
    def __init__(self, pais):
        self.pais = pais
    def obtener_pais(self):
        return self.pais

cuenta =
Cuenta(Cliente(Direccion("Argentina")))
pais = cuenta.obtener_pais_cliente()
```

Si Cumple!!

Tell don't ask

Se le puede pedir un valor de estado a un objeto (get) siempre que el dato no se utilice para tomar una decisión sobre el estado interno de dicho objeto

```
# Caso incorrecto: toma una decisión en base a un estado interno del objeto.
for factura in facturas:
    if not factura.esta_anulada():
        total_factura = factura.calcular_total()
        print(f"Total de la factura: {total_factura}")
    else:
        print("Total de la factura: 0")

# Caso correcto
for factura in facturas:
    total_factura = factura.calcular_total()
    print(f"Total de la factura: {total_factura}")
```

Tell don't ask

El siguiente código cumple con TDA?

```
class Pedido:
    def __init__(self, productos):
        self.productos = productos
    def obtener_total(self):
        return sum(producto.obtener_precio() for producto in
self.productos)

class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio
    def obtener_precio(self):
        return self.precio

class Carrito:
    def __init__(self, pedido):
        self.pedido = pedido
    def calcular_total(self):
        productos = self.pedido.productos
        total = sum(producto.obtener_precio() for producto in productos)
        descuento_aplicado = total * 0.9 if total > 100 else total
        return descuento_aplicado
```

Tell don't ask

El siguiente código cumple con TDA?

```
class Pedido:
    def __init__(self, productos):
        self.productos = productos
    def obtener_total(self):
        return sum(producto.obtener_precio() for producto in self.productos)

class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio
    def obtener_precio(self):
        return self.precio

class Carrito:
    def __init__(self, pedido):
        self.pedido = pedido
    def calcular_total(self):
        productos = self.pedido.productos
        total = sum(producto.obtener_precio() for producto in productos)
        descuento_aplicado = total * 0.9 if total > 100 else total
        return descuento_aplicado
```

NO Cumple!!

Único punto de salida

Un programa debe tener un punto de entrada principal y un único punto de salida donde la ejecución del programa termina.

```
def prueba(a):  
    if a%2 == 0: # Devolver el doble de a  
        return a*2  
    else: # a no es par, devolver lo mismo  
        return a
```

incorrecto

```
def prueba(a):  
    resultado = None  
    if a%2 == 0: # Devolver el doble de a  
        resultado = a*2  
    else: # a no es par, devolver lo mismo  
        resultado = a  
    return resultado
```

correcto

Patron Singleton

- Patron creacional
- Garantiza que la clase tenga una única instancia
- Proporciona acceso global a dicha instancia

Solo se inicializa cuando se llama por primera vez.

Evitar el acceso simultáneo a determinados recursos.

```
class Singleton(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super(Singleton, cls).__call__(*args, **kwargs)
        return cls._instances[cls]

class SingletonClass(metaclass=Singleton):
    pass

class RegularClass():
    pass

x = SingletonClass()
y = SingletonClass()
print(x == y)

x = RegularClass()
y = RegularClass()
print(x == y)
```

Patron Composite

- Patron estructural

Sugiere trabajar con una clase Composite y con clases Leaf que esten relacionadas a traves de una interfaz comun.

```
class Leaf(Component):  
  
    def operation(self) -> str:  
        return "Leaf"
```

```
class Composite(Component):  
    def __init__(self) -> None:  
        self._children: List[Component] = []  
    def operation(self) -> str:  
        results = []  
        for child in self._children:  
            results.append(child.operation())  
        return f"Branch({'+' .join(results)})"
```

Patron Observer

- Patron
Comportamiento

Debemos identificar
notificadores y
subscriptores.

```
class Publisher(Subject):  
    _state: int = None  
    _subscribers: List[Observer] = []  
  
    def notify(self) -> None:  
  
        print("Subject: Notifying observers...")  
        for observer in self._observers:  
            observer.update(self)
```

```
class Observer(ABC):  
    @abstractmethod  
    def update(self, subject: Subject) -> None:  
        """  
        Receive update from subject.  
        """  
        pass
```